Micro-SIEM – Viva Preparation Guide

Everything you need to explain the project: APIs, sessions, network access, OpenSearch, Vector, likely faculty questions and demo tips

```
Log files → Vector (parse + tag) → OpenSearch logs-index → FastAPI detection → alerts-index → Dashboards (refresh every 3 s)
```

Contents

1. API Endpoints – overview
2. API Endpoints – how each one works
3. Dashboard Sessions – token, not cookie
4. Using the Dashboard on the Same LAN
5. OpenSearch in this Project
6. Vector in this Project
7. Likely Faculty Questions with Answers
8. Demo-Day Tips

1. API Endpoints – Overview

The backend (`src/api/main.py`) has **12 API endpoints** in 4 groups. FastAPI also creates `/docs` , `/redoc` and `/openapi.json` automatically – those are not our code.

Group 1 – Health check (1)

Method	Endpoint	Auth	What it does
GET	`/`	none	Returns `{"status": "Micro-SIEM API running"}` – proves the API is up

Group 2 – Demo website, the attack target (2)

Method	Endpoint	Auth	What it does
POST	`/login`	none	Checks the demo login (admin / test / guest). **Every attempt is written to `login.log`** – one of the SIEM's log sources
GET	`/session-check`	demo token	Called by the search page on load – checks the demo session is valid and the account hasn't been blocked

Group 3 – Dashboard authentication (2)

Method	Endpoint	Auth	What it does
POST	`/auth/login`	none	Admin or viewer login – returns a random session token (valid 8 hours) and the role
POST	`/auth/logout`	token	Deletes the token immediately

Group 4 – SIEM data and actions (7)

Method	Endpoint	Who can call	What it does
GET	`/logs`	admin + viewer	Latest raw log events from OpenSearch
GET	`/alerts`	admin + viewer	**Runs the detection rules**, then returns alert history
GET	`/blocked`	admin + viewer	List of blocked IPs and accounts
POST	`/block/{ip}`	admin only	Block an IP with iptables – OFF returns 503
POST	`/unblock/{ip}`	admin only	Unblock an IP – OFF
POST	`/block-account/{username}`	admin only	Block a demo account – OFF
POST	`/unblock-account/{username}`	admin only	Unblock a demo account – OFF

Summary: 12 endpoints – 5 GET, 7 POST. 3 are public (`/` , `/login` , `/auth/login`); 9 need a token (`/session-check` the demo token, the other 8 a dashboard token). 4 of those are admin-only (viewer gets HTTP 403) and those same 4 are switched off for now (HTTP 503).

Viva tip: there is no separate "run detection" endpoint – detection runs *inside* `GET /alerts` , which the dashboard calls every 3 seconds. That is a known limitation; the better design is a background scheduler (listed in Future Scope).

2. API Endpoints – How Each One Works

1. GET / – health check

Called by: you or a browser to test. **Input:** nothing. **Returns:** `{"status": "Micro-SIEM API running"}` . No logic – just confirms FastAPI is up.

2. POST /login – demo website login (the attack target)

Called by: `nginx/html/login.html` through the `/api/login` proxy. **Input:** `{"username": "...", "password": "..."}`

1. Takes the client IP from the `X-Forwarded-For` header.
2. Checks `blocked-index` – if the account is blocked, logs `login_blocked` and refuses.

3. Compares the password with `DEMO_ACCOUNTS` (admin/1234, test/1111, guest/0000).
4. Writes `login_success` or `login_failed` as one JSON line to `/var/log/webapp/login.log` – **this is how web login attacks become SIEM data.**

Returns: success `{"success": true, "token": "..."}`; failure `{"success": false, "reason": "invalid credentials"}`.

3. GET /session-check?token=... – demo session guard

Called by: `search.html` on load. Checks the demo token exists in memory (`demo_sessions`) and that the account hasn't been blocked since login (if it has, the token is deleted). **Returns:** `{"valid": true, "username": "..."}` or `{"valid": false, "reason": "..."}`.

4. POST /auth/login – dashboard login

Called by: `react-dashboard/login.html`. Checks `DASHBOARD_USERS`, then creates a random 48-character token (`secrets.token_hex(24)`) stored in server memory with the role and an 8-hour expiry. **Returns:** `{"token": "...", "role": "admin"}`; wrong password → **HTTP 401**. The browser keeps the token in `sessionStorage` and sends it on every call as `Authorization: Bearer <token>`.

5. POST /auth/logout

Called by: the dashboards' Logout button. Deletes the token from memory so it stops working immediately. **Returns:** `{"status": "logged out"}`.

6. GET /logs?limit=50 – raw events

Auth: any valid token. Queries `logs-index`, newest first (dashboards request 200). **Returns:** `{"count": N, "logs": [...]}`. **Used for:** the log table, "Total Logs", the login-success doughnut and the activity line chart.

7. GET /alerts?limit=50 – the heart of the SIEM

Auth: any valid token. First **runs detection**, then returns alert history:

1. **Brute force:** 5 or more failures from one IP in 2 minutes (SSH and web) → T1110.
2. **Correlation:** 2 or more failures, then a success from the same IP → T1078.
3. **XSS:** a signature such as `<script` in an nginx request path → CWE-79, not repeated within 5 minutes.
4. New alerts are saved to `alerts-index`; the latest 50 are returned.

Used for: the alerts count, the Attack Vectors bar chart and the Threats table.

8. GET /blocked

Auth: any valid token. Lists up to 100 records from `blocked-index`, newest first. **Used for:** the "Blocked" panel on the admin Threats page.

9–12. Response actions (admin only, currently OFF)

Endpoint	Designed behaviour
`POST /block/{ip}`	Validates the IP, refuses loopback addresses, runs `iptables` (IPv4) or `ip6tables` (IPv6) with `-I INPUT -s <ip> -j DROP`, saves a record in `blocked-index`
`POST /unblock/{ip}`	Same command with `-D` to remove the rule; deletes the record
`POST /block-account/{username}`	Saves an account record, so `/login` refuses that user from then on
`POST /unblock-account/{username}`	Deletes the account record

Right now all four return **HTTP 503** "Blocking is disabled – coming in a future version" because `BLOCKING_ENABLED = False`. A viewer gets **HTTP 403** first – the role check runs before the disabled check.

> **Key point for the viva:** `/login` creates the evidence → Vector ships it to OpenSearch → `/alerts` detects the attack → the dashboard shows it.

3. Dashboard Sessions – Token, Not Cookie

The project uses **token-based sessions, not cookies**. The server keeps each session in memory; the browser keeps the token in `sessionStorage`.

How it works

1. **Login:** the dashboard sends username and password to `POST /auth/login`.
2. **Server creates the session** (`main.py`):

```
token = secrets.token_hex(24)          # random 48-char token
sessions[token] = {"role": "admin", "expires": now + 8 hours}
```

3. **Browser stores the token** (`login.html`):

```
sessionStorage.setItem("siem_token", data.token);
sessionStorage.setItem("siem_role", data.role);
```

4. **Every API call sends it in a header** (not a cookie): `Authorization: Bearer <token>`
5. **Server checks it** in `require_session()`: missing/unknown token → 401; expired → 401 and the session is deleted; `require_admin()` → 403 if the role is not admin.
6. **Logout:** `POST /auth/logout` deletes the token on the server and the browser runs `sessionStorage.clear()`.

The demo website works the same way with a separate `demo_token` in sessionStorage and a separate `demo_sessions` dictionary.

Why sessionStorage instead of a cookie

- The dashboard (port 8090) and API (port 8000) are on different ports – a Bearer header works with simple CORS; cookies would need extra `credentials` / `SameSite` setup.
- The token is **cleared when the tab closes** – suits a security console.
- Browsers don't send it automatically, so **CSRF** is not a risk.

Weaknesses (be ready for these)

Weakness	Explanation	Better option
Readable by JavaScript	If the dashboard ever had an XSS bug, a script could read the token from sessionStorage	`HttpOnly` cookie (JavaScript can't read it)
Sessions in memory	Restarting the API logs everyone out	Redis or a database
Plain HTTP	The token travels unencrypted on the network	HTTPS / TLS
Fixed passwords in code	`DASHBOARD_USERS` is hardcoded	Hashed passwords (bcrypt) in a database

One-line viva answer: "We use server-side, token-based sessions. On login the server issues a random token valid for 8 hours. The browser keeps it in sessionStorage and sends it as a Bearer token on every request, and the server checks the token and the role on every call. No cookies are used."

4. Using the Dashboard on the Same LAN

Other devices (phone, second PC) on the same Wi-Fi can open the dashboard and demo site using your PC's LAN IP. Find it with `hostname -I` or `ip -4 addr` (at the time of writing it was **10.131.74.123** – it can change when you reconnect).

1. On your PC: `cd ~/micro-siem && docker-compose up -d`
2. Connect the other device to the **same Wi-Fi**.
3. Open on that device:

What	URL
Dashboard (admin / viewer login)	`http://<your-IP>:8090`
Demo website (attack target)	`http://<your-IP>:8080`
API docs	`http://<your-IP>:8000/docs`

Why it works without any config change

- Docker publishes ports 8090 and 8080 on **all network interfaces**, not only localhost.
- The dashboard finds the API automatically: `const API_URL = \`http://${window.location.hostname}:8000\`;` – opened at `<IP>:8090`, it calls `<IP>:8000`. The API runs in host network mode, so port 8000 is open on your PC's IP.

Best demo setup

- **PC (projector):** admin dashboard.
- **Phone:** attack the demo site – 6 wrong passwords, then the right one, then an XSS search.
- **Second laptop (optional):** viewer login to show read-only access.

Alerts then show the **phone's real IP**, not `172.18.0.1` (the Docker gateway you get when attacking from the same PC).

If it doesn't open on the other device

Problem	Fix
Page doesn't load at all	College / public Wi-Fi often blocks devices from talking to each other ("client isolation"). Use your phone's hotspot and connect both devices to it.
Dashboard loads but no data	A firewall may block port 8000: check `sudo ufw status`; if active, `sudo ufw allow 8000,8080,8090/tcp`
IP changed	Check again with `hostname -I`

Test this on the college network before demo day and keep the phone-hotspot option ready.

5. OpenSearch in this Project

What it is

OpenSearch is an open-source **search and analytics engine** (a community fork of Elasticsearch, maintained by AWS and others). It stores data as **JSON documents** and can search millions of them in milliseconds. In Micro-SIEM it is the **central storage – the SIEM's memory**: every log, alert and block record lives there.

```
Vector ──(writes logs)──► OpenSearch ◄──(reads logs, writes alerts)── FastAPI ──► Dashboards
```

How it's set up (docker-compose.yml)

```
opensearch:
  image: opensearchproject/opensearch:latest
  environment:
    - discovery.type=single-node                 # one server, no cluster
    - DISABLE_SECURITY_PLUGIN=true               # no login/TLS (lab only)
    - "OPENSEARCH_JAVA_OPTS=-Xms512m -Xmx512m"   # 512 MB Java memory
  ports:
    - "9200:9200"                                # REST API
```

- **Port 9200** – REST API; Vector and FastAPI talk to it over plain HTTP.
- **Single node** – cluster health shows **yellow**, which is normal (no second node for replica copies).
- **OpenSearch Dashboards** (port 5601) – separate container for browsing raw data (Figure 6.10).

The three indices (index = table, document = row)

Index	Written by	Contains	Example fields
`logs-index`	Vector	Every log event from the 3 sources	timestamp, log_type, source_ip, event, username, path
`alerts-index`	FastAPI (detection)	Every alert raised	alert_type, source_ip, attempt_count, severity, mitre_id
`blocked-index`	FastAPI (block actions)	Blocked IPs and accounts	type, ip, username, blocked_at

No schema is defined: OpenSearch uses **dynamic mapping** – it works out field types from the first document. Every text field also gets a `.keyword` copy stored exactly, which is why the code uses `username.keyword` for exact matches.

How data gets in (Vector)

```
[sinks.opensearch_logs]
type = "elasticsearch"                # OpenSearch speaks the same API
endpoints = ["http://opensearch:9200"]
mode = "bulk"                         # many events per request
api_version = "v8"                    # without this → 400 error (old _type field)
```

How FastAPI queries it (the detection logic)

Query	Used for	Example from the code
`match`	Text search	`{"match": {"event": "login_failed"}}`
`range`	Time window	`{"range": {"timestamp": {"gte": <2 min ago>}}}`
`term` + `.keyword`	Exact match	`{"term": {"username.keyword": "test"}}`
`bool → must`	Combine conditions (AND)	failed logins **and** last 2 minutes
`sort`	Newest first	`{"timestamp": {"order": "desc"}}`

Brute-force detection in one sentence: search `logs-index` for `login_failed` events from the last 2 minutes, group them by `source_ip` in Python, and if an IP has 5 or more, write an alert to `alerts-index`.

Why OpenSearch and not MySQL?

Need	OpenSearch	Relational DB (MySQL)
Different log formats	Schema-free JSON	A fixed table per format
Fast time-range searches	Built for it	Slow on large logs
Full-text search on messages	Built in	Limited
Industry use	Same approach as ELK and Wazuh	Not used for SIEMs
Cost	Free, open source	—

Weaknesses (be honest)

- **No data volume:** `docker-compose stop` / `start` keeps data, but **`docker-compose down` deletes all logs and alerts**. Fix: a named volume on `/usr/share/opensearch/data`.
- **Security plugin off:** anyone on the network can read or change data on port 9200 – fine for a lab, not production.
- **No retention:** `logs-index` grows forever; production uses lifecycle policies or daily indices (`logs-2026.09.26`).
- **Single node:** no redundancy if it crashes.

> **One-line viva answer:** "OpenSearch is our central log store and search engine. Vector bulk-inserts parsed logs into logs-index; our FastAPI detection engine queries it with time-range and match queries, writes alerts into alerts-index, and the dashboards read from it. We chose it because it's schema-free, fast for time-based search and the industry-standard approach used in ELK and Wazuh."

6. Vector in this Project

What it is

Vector is an open-source, high-performance **log collection agent** written in Rust (originally by Timber.io, now maintained by Datadog). Its job: **read logs, clean and structure them, ship them somewhere**. In Micro-SIEM it is the **bridge between the log files and OpenSearch** – the "collect + parse" stage. Without Vector the logs would just sit in files.

```
login.log ──────┐
access_json.log ┼──► VECTOR (read → parse → tag) ──► OpenSearch logs-index
auth.log ───────┘
```

The three-step model (vector/vector.toml)

Step	Vector term	In this project
1. Where data comes from	**source**	3 log files
2. How to change it	**transform**	VRL code adds `log_type` and `source_ip`
3. Where it goes	**sink**	OpenSearch (+ console for debugging)

Pipeline 1 – SSH auth.log (plain text, needs parsing)

```
[sources.auth_log]
type = "file"
include = ["/var/log/auth.log"]
read_from = "end"            # only new lines, not old history

[transforms.parse_auth]
type = "remap"
source = '''
.log_type = "linux_auth"
.raw_message = .message
parsed, err = parse_regex(.message, r'from (?P<ip>[0-9a-fA-F:.]+) port')
if err == null { .source_ip = parsed.ip } else { .source_ip = "unknown" }
'''
```

Input: `Failed password for rabbit from 192.168.1.5 port 55224 ssh2` → the regex extracts `source_ip = "192.168.1.5"` (works for IPv4 and IPv6). This is **real parsing** – unstructured text becomes fields.

Pipeline 2 – web login log (already JSON)

```
[transforms.parse_webapp]
source = '''
parsed, err = parse_json(.message)
if err == null { . = merge!(., parsed) }
'''
```

FastAPI writes each login attempt as a JSON line; Vector parses it and merges the fields (`event` , `username` , `source_ip` …) into the event.

Pipeline 3 – Nginx access log (JSON by design)

The nginx config was deliberately changed to write JSON logs (`log_format json_combined` in `nginx/default.conf`), so Vector parses them the same simple way. **Design point:** making the source produce structured logs avoids fragile regex parsing.

The sinks

```
[sinks.opensearch_logs]
type = "elasticsearch"
inputs = ["parse_auth", "parse_webapp", "parse_nginx"]
endpoints = ["http://opensearch:9200"]
mode = "bulk"                # batches events for speed
api_version = "v8"           # needed for new OpenSearch

[sinks.debug_console]
type = "console"             # prints every event → docker-compose logs vector
```

All three pipelines go to **one index** (`logs-index`), told apart by `log_type` .

How Vector reaches the log files (Docker volumes)

Log	Written by	Shared via
`login.log`	fastapi-app container	`webapp-logs` volume
`access_json.log`	nginx container	`nginx-logs` volume
`auth.log`	Linux host	bind mount `/var/log/auth.log`

All mounts are **read-only (`:ro`)** for Vector – it can read logs but never change them (log integrity).

Real bugs fixed with Vector (good viva material)

Bug	Cause	Fix
Bulk insert HTTP 400	New OpenSearch rejects the old `_type` field	`api_version = "v8"`
VRL compile error	`merge()` can fail and VRL forces you to handle it	`merge!()` (raise-on-error version)
`auth.log` became an empty directory	Docker created the mount path before rsyslog existed	Removed the directory, installed rsyslog
Queries on `@timestamp` found nothing	Vector's field is `timestamp`	Changed the queries

Why Vector and not the alternatives?

Tool	Why not
Logstash	Java-based, heavy (about 1 GB RAM)
Filebeat	Ships logs but has limited transformation
Fluentd	Ruby-based, slower, plugin-heavy
Vector	Single fast Rust binary, low memory, and a real language (VRL) for parsing

Weaknesses

- **No persistent data directory:** Vector's read checkpoints live inside the container; if it is recreated, with `read_from = "end"` it may **skip lines** written in between.
- **Unpinned image** (`latest-alpine`): an update could change behaviour before the demo.
- **Silent parse failures:** a broken JSON line still goes in unparsed, with no alert.

One-line viva answer: "Vector is our log collector. It tails the three log files, uses VRL transforms to parse them into structured JSON with log_type and source_ip – regex for SSH, JSON parsing for web and Nginx – and bulk-inserts them into OpenSearch. We chose it because it's lightweight, fast and lets us write real parsing logic."

7. Likely Faculty Questions with Answers

A. Basic concepts (almost certain)

1. What is a SIEM?

Security Information and Event Management – collects logs from many sources, stores them centrally, runs detection rules and raises alerts for analysts. Examples: Splunk, IBM QRadar, Microsoft Sentinel, Wazuh.

2. SIEM vs IDS vs firewall?

Firewall blocks or allows traffic by rules (prevents). **IDS** inspects network traffic for attacks as it happens. **SIEM** collects logs from many systems and correlates them – it sees the bigger picture, e.g. many failures followed by a success.

3. What is a SOC?

Security Operations Centre – the team that watches SIEM dashboards around the clock and responds to alerts. Our admin dashboard is a mini SOC console.

4. What is MITRE ATT&CK?

A global knowledge base of real attacker techniques with IDs. We use **T1110** (Brute Force) and **T1078** (Valid Accounts).

5. Why is XSS mapped to CWE-79 and not MITRE?

ATT&CK describes attacker *behaviour*; XSS is a web-application *vulnerability class*, described by OWASP and CWE. Forcing it into ATT&CK would be wrong – the mapping is honest.

B. Your design (very likely)

6. Explain the data flow.

Log file → Vector (parse and tag) → OpenSearch logs-index → FastAPI detection → alerts-index → dashboard, refreshed every 3 seconds.

7. Threshold vs correlation rule?

Threshold counts events (5 or more failures in 2 minutes). **Correlation** looks at a *sequence* across events (failures then a success from the same IP) – the attacker probably got in, which is more dangerous than failures alone.

8. Why 5 attempts and 2 minutes?

A normal user mistypes 1–2 times; 5 in 2 minutes is clearly automated or deliberate guessing, and easy to trigger in a demo. In production it would be tuned to reduce false positives.

9. What is a false positive? Can your system create one?

An alert without a real attack. Yes – e.g. many users behind one college NAT IP mistyping passwords could trigger brute force. Reduced by tuning thresholds and allowlists.

10. Why Docker?

Runs anywhere with one command, isolates each component, avoids "works on my machine" problems, and is the industry standard for deployment.

11. Why does the API use `network_mode: host` and `NET_ADMIN`?

IP blocking must change the **host's** firewall (iptables) because SSH runs on the host, not in a container. Host network plus the NET_ADMIN capability allows that.

12. Why is blocking disabled?

It was built and tested (including IPv6) but is switched off by a flag and planned for a future version – automatic firewall changes can lock out real users and need more safeguards. Also, web traffic to Docker ports goes through Docker's own firewall chain (`DOCKER-USER`), not `INPUT`, so the web part needs more work.

13. Why is the search page deliberately vulnerable?

It is a safe, self-owned target so XSS can be demonstrated live and the SIEM shown detecting it. No third-party system is attacked – ethical and legal.

C. Technical questions

14. What are FastAPI, REST, Uvicorn and Pydantic?

FastAPI – Python framework for REST APIs. **REST** – HTTP methods (GET/POST) acting on URLs. **Uvicorn** – the ASGI server that runs FastAPI. **Pydantic** – validates request JSON.

15. How is authentication done?

Server-side token sessions: a random token valid 8 hours, stored in sessionStorage, sent as a Bearer header and checked on every call. Roles are enforced on the server – a viewer gets 403.

16. What is CORS and why is it enabled?

Browsers block calls to a different origin by default. The dashboard (port 8090) calls the API (port 8000), so CORS must allow it.

17. What happens if OpenSearch goes down?

Vector buffers and retries, so logs aren't lost at first; the API returns errors and the dashboard shows stale data until it comes back.

18. How do you get the attacker's real IP behind nginx?

From the `X-Forwarded-For` header that nginx sets – see question 21 for the follow-up.

D. Tricky questions (prepare these)

19. What is your own work and what came from tools?

Tools: OpenSearch, Vector, Docker, nginx. **Own work:** the detection engine (all 4 rules, correlation, de-duplication), the Vector parsing rules, the API and auth/role system, both dashboards, the demo attack site, blocking logic with IPv4/IPv6 detection, and the integration of everything.

20. Can the SIEM itself be attacked?

Yes – OpenSearch has no security plugin, passwords are hardcoded and there is no TLS. Acceptable for a lab; all of it is in limitations and future scope.

21. Can an attacker fake their IP?

Yes – the IP comes from `X-Forwarded-For`, which a client can set. The fix is to trust only the header nginx itself sets (`X-Real-IP`). Admitting this shows understanding.

22. Why not machine learning?

Rule-based detection is transparent and explainable – what a SOC needs to trust alerts. ML needs large labelled datasets and produces unexplainable alerts. Possible future scope.

23. Why does the alert count grow fast during an attack?

Brute-force alerts are not de-duplicated like XSS alerts, so an ongoing attack adds an alert on each refresh. Known limitation; the dashboard groups them by IP for display.

24. How would this scale to 1000 servers?

Vector on each server, an OpenSearch cluster with multiple nodes, detection moved to a background scheduler, daily indices with retention. The architecture stays the same – only scale changes.

E. Closing questions

25. What did you learn?

Log management, detection engineering, MITRE ATT&CK, REST APIs, search engines, Docker and secure authentication design.

26. What would you add next?

Password-spraying detection (T1110.003), enabling response actions, e-mail/Telegram notifications, GeoIP enrichment and TLS.

8. Demo-Day Tips

1. **Test on the college network beforehand.** If devices can't see each other, use your phone's hotspot.
2. **Keep screenshots ready** as a backup – they are already in the report and slides.
3. **Attack from your phone** so a real IP appears, not `172.18.0.1`.
4. **Admit limitations confidently.** "I know this weakness and here's the fix" scores better than pretending it's perfect.

Live demo order

1. Start: `cd ~/micro-siem && docker-compose up -d` – wait about a minute.
2. PC: open the admin dashboard `http://<IP>:8090` and log in as admin.
3. Phone: open `http://<IP>:8080/login.html` and enter a wrong password 6 times → **Web Login Brute Force (T1110)** appears.
4. Phone: 2 wrong passwords, then the correct one (e.g. admin / 1234) → **Suspicious Login (T1078)**.
5. Phone: on the search page, search `<script>alert(1)</script>` → **XSS Attempt Detected (CWE-79)**.
6. PC: show the Attack Vectors chart (red = brute force / XSS, blue = suspicious login), the Threats table with MITRE IDs, and the "Coming Soon" pop-up on Block.
7. Optional: log in as viewer on a second device to show read-only access.

Useful commands

Command	Purpose
`docker-compose up -d`	Start all 6 containers
`docker-compose ps`	Check every container is Up
`docker-compose logs -f vector`	Watch events flowing (debug console sink)
`docker-compose logs -f api`	Watch API requests and errors
`docker-compose stop`	Stop but keep data
`docker-compose down`	Stop and remove containers – **deletes OpenSearch data**
`hostname -I`	Find your LAN IP